

GUÍA DE TRABAJO PARA ESTUDIANTES

Secciones 5 y 6

React SPA, componentes y navegación + Formularios, validación y servicios API con Laravel

Continuidad del proyecto de nivel intermedio

Propósito general

Continuar el proyecto que cada equipo viene desarrollando. En la Sección 5 se reorganizará el frontend como una SPA modular con componentes, layouts, rutas y permisos. En la Sección 6 se conectará esa estructura con formularios controlados, validación, servicios HTTP y un CRUD REST completo contra Laravel.

Regla de trabajo

Los ejemplos utilizan el módulo Catálogos porque ya forma parte del proyecto trabajado. Cada equipo debe adaptar nombres, campos, permisos y rutas al módulo real de su sistema, sin perder la estructura explicada en la guía.

1. Punto de partida del proyecto

Antes de iniciar esta guía, el proyecto debe contar con backend Laravel, API, autenticación, roles, permisos y al menos un CRUD de una tabla catálogo. A partir de aquí el objetivo es ordenar el frontend y luego completar la integración React ↔ Laravel.

- ☐ El backend Laravel inicia correctamente.
- ☐ El frontend React inicia correctamente.
- ☐ El login y la sesión ya funcionan.
- ☐ El sistema maneja roles y permisos.
- ☐ Existe al menos un módulo CRUD de catálogo.
- ☐ El equipo puede identificar qué rutas son públicas y cuáles requieren sesión.

Idea central

React organiza la experiencia del usuario: páginas, componentes, navegación y estado. Laravel sigue siendo la autoridad sobre autenticación, autorización, reglas y persistencia de datos.

PARTE I · SECCIÓN 5

React SPA: componentes, sintaxis y navegación

2. Objetivos de trabajo de la Sección 5

- Comprender la diferencia entre una SPA y una navegación con recarga completa.
- Dividir la interfaz en componentes con responsabilidades claras.
- Usar JSX para representar datos, eventos, condiciones y listas.
- Diferenciar props y state.
- Configurar React Router para relacionar URL, página, layout y parámetros.
- Integrar navegación protegida y permisos sin sustituir la autorización de Laravel.
- Reorganizar el frontend del proyecto para que nuevos módulos sigan una estructura consistente.

Resultado esperado

Al finalizar la Sección 5, el frontend debe tener una estructura clara de carpetas, navegación funcional y al menos un módulo organizado en páginas y componentes reutilizables.

3. Paso 1 — Revisar la estructura actual del frontend

Revise el contenido de src/. Si App.jsx contiene menú, formulario, tabla, permisos, peticiones y lógica de navegación en un solo archivo, debe separarse.

Estructura de referencia:

```
src/
├── components/
│   ├── ui/
│   └── catalogos/
├── pages/
│   ├── DashboardPage.jsx
│   └── CatalogosPage.jsx
├── layouts/
│   └── DashboardLayout.jsx
├── routes/
│   └── AppRoutes.jsx
├── services/
│   └── api.js
├── context/
│   └── AuthContext.jsx
├── hooks/
└── App.jsx
```

Carpeta	Responsabilidad
pages/	Pantallas asociadas a rutas; una página compone varios componentes.
components/	Piezas reutilizables como tablas, formularios, botones, modales y tarjetas.
layouts/	Estructuras compartidas como sidebar, header y área de contenido.
services/	Comunicación HTTP con Laravel.
context/	Estado transversal como usuario autenticado y permisos.
routes/	Relación entre URL, página, layout y protección.

Actividad

Cree las carpetas que todavía no existan y mueva gradualmente los archivos. No rompa primero todo el proyecto para después repararlo: haga cambios por bloques y pruebe la aplicación después de cada movimiento.

4. Paso 2 — Comprender el modelo de una SPA

En una SPA el navegador mantiene cargada la aplicación React. Cuando la URL cambia, React Router determina qué página o componente debe mostrarse sin solicitar un documento HTML completo en cada navegación.

1. El usuario entra a una URL, por ejemplo /catalogos.
2. React Router busca la ruta que coincide.
3. Se renderiza la página correspondiente, por ejemplo CatalogosPage.
4. La página solicita datos al backend Laravel mediante la API.

Comprobación

La URL debe seguir representando dónde está el usuario. Eso permite historial del navegador, favoritos y acceso directo a rutas del proyecto.

5. Paso 3 — Separar una página en componentes

Una página de catálogo puede coordinar datos y eventos, mientras delega la interfaz a componentes especializados.

Componente	Responsabilidad
CatalogosPage	Coordina datos, estado y eventos del módulo.
CatalogoForm	Captura y modifica datos.
CatalogoTable	Representa la lista.
ConfirmDialog	Confirma una acción destructiva.

Regla docente

Un componente debería tener un propósito que pueda explicarse en una frase.

6. Paso 4 — JSX, props y estado

JSX permite combinar JavaScript y marcado. Las expresiones se escriben entre llaves { }. Las props llegan desde el componente padre y el state pertenece al componente que necesita recordar cambios entre renderizados.

6.1 Ejemplo de JSX con props

```
export default function CatalogoCard({ item, onEdit }) {
  const estado = item.activo ? 'Activo' : 'Inactivo';

  return (
    <article className="card">
      <h3>{item.nombre}</h3>
      <p>{estado}</p>

      {item.activo && (
        <button onClick={() => onEdit(item)}>
          Editar
        </button>
      )}
    </article>
  );
}
```

```

    })
  </article>
);
}

```

6.2 Ejemplo de estado con useState

```

import { useState } from 'react';

export default function FiltroCatalogo() {
  const [texto, setTexto] = useState('');

  return (
    <input
      value={texto}
      onChange={(e) => setTexto(e.target.value)}
      placeholder="Buscar..."
    />
  );
}

```

Pregunta para decidir

¿El valor viene configurado por el padre? → prop. ¿Debe cambiar por interacción y provocar una actualización visual? → probablemente state.

7. Paso 5 — Renderizado condicional y listas

La misma página puede mostrar contenidos distintos según carga, errores, permisos o existencia de datos.

```

{loading && <Spinner />}

{error && <Alert message={error} />}

{!loading && items.length === 0 && (
  <EmptyState text="No hay registros" />
)}

{items.map((item) => (
  <CatalogoRow
    key={item.id}
    item={item}
  />
))}

{can('catalogos.create') && (
  <button>Nuevo</button>
)}

```

- Use &&, ternarios o funciones auxiliares para mostrar contenido según una condición.
- Use map() para convertir colecciones en componentes.
- Cada elemento de una lista debe tener una key estable; normalmente el id.
- Ocultar una acción por permiso mejora la experiencia, pero Laravel sigue verificando la autorización real.

8. Paso 6 — Instalar y configurar React Router

Instale React Router y envuelva la aplicación con BrowserRouter.

```

npm i react-router

import { BrowserRouter } from 'react-router';

createRoot(root).render(

```

```

<BrowserRouter>
  <App />
</BrowserRouter>
);

```

Concepto

BrowserRouter utiliza el historial del navegador para sincronizar la URL con la navegación de la SPA.

9. Paso 7 — Crear las rutas del proyecto

Cree routes/AppRoutes.jsx. La URL debe representar módulos y acciones importantes del sistema.

```

import { Routes, Route } from 'react-router';

<Routes>
  <Route path="/login" element={<LoginPage />} />

  <Route path="/" element={<ProtectedRoute />>
    <Route element={<DashboardLayout />>
      <Route index element={<DashboardPage />} />
      <Route path="catalogos" element={<CatalogosPage />} />
      <Route path="catalogos/:id" element={<CatalogoEditPage />} />
      <Route path="roles" element={<RolesPage />} />
      <Route path="usuarios" element={<UsuariosPage />} />
    </Route>
  </Route>

  <Route path="*" element={<NotFoundPage />} />
</Routes>

```

- /login es una ruta pública.
- ProtectedRoute comprueba la sesión antes de mostrar el árbol privado.
- catalogos/:id usa un parámetro de URL.
- path='*' atiende las URLs no reconocidas.

10. Paso 8 — Crear un layout con <Outlet />

El layout evita repetir sidebar, header y contenedor en cada página. Las rutas hijas se muestran dentro de <Outlet />.

```

import { Outlet } from 'react-router';

export default function DashboardLayout() {
  return (
    <div className="app-shell">
      <Sidebar />

      <section className="main-area">
        <Header />
        <main>
          <Outlet />
        </main>
      </section>
    </div>
  );
}

```

Resultado

Al navegar entre Dashboard, Catálogos, Roles y Usuarios cambia el contenido central, mientras el layout general permanece.

11. Paso 9 — Rutas dinámicas y parámetros

Use parámetros para editar o consultar recursos sin crear una ruta distinta por cada registro.

```
<Route
  path="/catalogos/:id"
  element={<CatalogoEditPage />}
/>

import { useParams } from 'react-router';

const { id } = useParams();
```

5. El usuario selecciona Editar.
6. La URL cambia, por ejemplo, a /catalogos/12.
7. useParams() obtiene id = 12.
8. La página puede solicitar GET /api/catalogos/12.

Seguridad

No coloque información sensible en la URL. Laravel debe verificar la autorización aunque el usuario conozca o modifique el id.

12. Paso 10 — Navegación con Link, NavLink y useNavigate

La navegación interna debe conservar el comportamiento SPA. No use window.location para navegación normal entre pantallas del proyecto.

```
<NavLink
  to="/catalogos"
  className={({ isActive }) =>
    isActive ? 'menu active' : 'menu'
  }
>
  Catálogos
</NavLink>

const navigate = useNavigate();

async function guardar() {
  await catalogosApi.create(form);
  navigate('/catalogos');
}
```

- Link: navegación declarativa básica.
- NavLink: ideal para menús porque permite conocer si la ruta está activa.
- useNavigate: útil después de una acción, por ejemplo guardar y regresar al listado.

13. Paso 11 — Aplicar permisos a la navegación

El menú puede adaptarse al usuario. La interfaz oculta opciones que no corresponden, pero Laravel sigue siendo responsable de impedir acciones no autorizadas.

```
const menu = [
  { to: '/', label: 'Inicio' },
  {
    to: '/catalogos',
    label: 'Catálogos',
    permiso: 'catalogos.view'
  },
  {
    to: '/roles',
```

```

    label: 'Roles',
    permiso: 'roles.view'
  }
];

menu
  .filter(item => !item.permiso || can(item.permiso))
  .map(item => <NavLink key={item.to} {...item} />);

```

Regla

Frontend orienta. Backend autoriza.

14. Trabajo de la Sección 5 sobre el proyecto

- ☐ Separar pages, components, layouts, services, routes y context.
- ☐ Crear Sidebar + Header + Outlet para el área autenticada.
- ☐ Registrar login, dashboard, catálogo y al menos una ruta administrativa.
- ☐ Dividir el CRUD en tabla, formulario y diálogo de confirmación.
- ☐ Mostrar navegación y acciones según los permisos del usuario.
- ☐ Probar URL directa, rutas privadas, menú activo, parámetros, permisos y página 404.

15. Pruebas mínimas de la Sección 5

Prueba	Acción	Resultado esperado
T1 · URL directa	Abrir /catalogos directamente.	La página aparece si existe sesión.
T2 · Sin sesión	Abrir una ruta privada.	Redirección al login.
T3 · Menú activo	Cambiar de módulo.	NavLink refleja la ruta actual.
T4 · Parámetro	Abrir /catalogos/ID.	La página recibe el parámetro correcto.
T5 · Permisos	Probar usuario sin permiso.	No ve el acceso y la API tampoco permite la acción.
T6 · 404	Escribir una URL inexistente.	Se muestra NotFound.

Evidencia de la Sección 5

Capturas de la navegación, estructura de carpetas y una explicación breve de cómo una URL llega al componente o layout que se muestra. Cada integrante debe poder enseñar una ruta en el código y explicarla.

PARTE II · SECCIÓN 6

Formularios, validación y servicios API

16. Objetivos de trabajo de la Sección 6

- Controlar formularios mediante estado React y eventos.
- Diferenciar validación de experiencia en React y validación obligatoria en Laravel.
- Centralizar solicitudes HTTP en una capa de servicios.
- Mostrar estados de carga, guardado, error, éxito y estado vacío.
- Completar crear, listar, editar y eliminar desde la SPA contra la API.
- Mantener los permisos del frontend coordinados con la autorización del backend.

Resultado esperado

Cada equipo debe llevar un módulo existente hasta un CRUD completo y consistente, conectado de React a Laravel y con comportamiento correcto ante errores.

17. Paso 1 — Convertir el formulario en controlado

Un input es controlado cuando su valor visible proviene del estado React y onChange actualiza ese estado.

```
const [form, setForm] = useState({
  nombre: '',
  descripcion: '',
  activo: true
});

function handleChange(e) {
  const { name, value, type, checked } = e.target;

  setForm(prev => ({
    ...prev,
    [name]: type === 'checkbox' ? checked : value
  }));
}

<input
  name="nombre"
  value={form.nombre}
  onChange={handleChange}
/>
```

- El estado es la fuente de verdad del campo.
- Se crea un nuevo objeto con ...prev en lugar de modificar el anterior.
- El atributo name permite reutilizar el mismo handler en varios inputs.

18. Paso 2 — Separar el estado del formulario

No mezcle valores editables, errores y estados de operación dentro del mismo objeto.

```
const [form, setForm] = useState(initialForm);
const [errors, setErrors] = useState({});
const [saving, setSaving] = useState(false);
const [message, setMessage] = useState('');
```

- form: valores que el usuario edita.
- errors: errores por campo.

- saving/loading/deleting: estado de la operación.
- message/generalError: mensajes generales.

Principio

No guarde en state información que puede calcularse a partir de otros valores. Por ejemplo, un total puede calcularse durante el render si no necesita edición independiente.

19. Paso 3 — Validación en dos capas

React ayuda a corregir antes de enviar. Laravel acepta o rechaza la solicitud. La base de datos conserva los datos válidos.

Capa	Responsabilidad
React	Mejora UX: requerido, mensajes inmediatos, formatos y prevención de envíos incompletos.
Laravel	Valida nuevamente cada solicitud y decide si puede pasar a la persistencia.

Regla de seguridad

Nunca confíe únicamente en la validación de React. Un cliente externo puede llamar directamente a la API sin usar el formulario.

20. Paso 4 — Crear un Form Request en Laravel

Para un proyecto intermedio, las reglas repetidas pueden moverse a un Form Request para mantener el controlador enfocado en la operación.

```
php artisan make:request StoreCatalogoRequest
class StoreCatalogoRequest extends FormRequest
{
    public function authorize(): bool
    {
        return $this->user()->can('catalogos.create');
    }

    public function rules(): array
    {
        return [
            'nombre' => ['required', 'string', 'max:150'],
            'descripcion' => ['nullable', 'string'],
            'activo' => ['required', 'boolean'],
        ];
    }
}
```

- authorize(): decide si el usuario puede realizar la solicitud.
- rules(): define las reglas que deben cumplir los datos.
- Cuando una solicitud espera JSON y falla la validación, Laravel devuelve errores estructurados con código 422.

21. Paso 5 — Crear una capa de servicios HTTP

La página no debe repetir URL, headers, JSON.stringify y comprobación de respuesta en cada acción.

```
const API_URL = import.meta.env.VITE_API_URL;

async function request(path, options = {}) {
  const response = await fetch(API_URL + path, {
    credentials: 'include',
    headers: {
      Accept: 'application/json',
      'Content-Type': 'application/json',
      ...options.headers,
    },
    ...options,
  });

  const data = response.status === 204
    ? null
    : await response.json();

  if (!response.ok) {
    const error = new Error(data?.message || 'Error');
    error.status = response.status;
    error.errors = data?.errors || {};
    throw error;
  }

  return data;
}
```

- Centraliza URL, credenciales, headers y lectura de respuestas.
- Normaliza errores con status y errors.
- Permite reutilizar la misma lógica en varios módulos.

22. Paso 6 — Definir las operaciones CRUD del servicio

Cada acción del CRUD debe usar el método HTTP que corresponde a la intención.

Operación	Método	Ejemplo de ruta
Listar	GET	/catalogos
Consultar uno	GET	/catalogos/:id
Crear	POST	/catalogos
Actualizar	PATCH / PUT	/catalogos/:id
Eliminar	DELETE	/catalogos/:id

23. Paso 7 — Crear registros desde React

El formulario construye una solicitud; Laravel autentica, autoriza, valida y finalmente persiste.

```
async function guardar(e) {
  e.preventDefault();
  setSaving(true);
  setErrors({});

  try {
    await catalogosApi.create(form);
    setForm(initialForm);
    await cargar();
  } catch (error) {
    setErrors(error.errors || {});
  } finally {
    setSaving(false);
  }
}
```

9. React ejecuta handleSubmit.
10. El servicio envía POST + JSON.

11. Laravel comprueba autenticación/autorización y valida.
12. El modelo guarda la información.
13. La API responde y React actualiza la interfaz.

24. Paso 8 — Cargar datos con useEffect

useEffect permite sincronizar el componente con un sistema externo, en este caso la API.

```
const [items, setItems] = useState([]);
const [loading, setLoading] = useState(true);
const [error, setError] = useState('');

useEffect(() => {
  let ignore = false;

  async function load() {
    try {
      setLoading(true);
      const data = await catalogosApi.list();
      if (!ignore) setItems(data);
    } catch (e) {
      if (!ignore) setError(e.message);
    } finally {
      if (!ignore) setLoading(false);
    }
  }

  load();
  return () => { ignore = true; };
}, []);
```

Nota

En desarrollo, Strict Mode puede ejecutar un ciclo extra para detectar efectos mal implementados. No lo desactive solo para ocultar este comportamiento.

25. Paso 9 — Mostrar estados de interfaz

Una UI profesional debe comunicar lo que está ocurriendo. No debe existir únicamente el estado “hay datos” o “no hay datos”.

Estado	Qué comunica
LOADING	GET pendiente: Cargando registros...
EMPTY	GET correcto pero colección vacía.
ERROR	Falló la solicitud o el backend respondió error.
SAVING	POST/PUT/PATCH en curso; evite acciones duplicadas.
SUCCESS	El backend confirmó la operación.
FORBIDDEN	El backend respondió 403 o la acción no corresponde al usuario.

26. Paso 10 — Interpretar errores HTTP

Código	Significado	Respuesta de la UI
401	No autenticado	Limpiar contexto y redirigir al login.
403	No autorizado	Informar que el usuario no posee permiso.
404	No encontrado	Indicar que recurso o ruta no existe.
422	Validación	Mostrar errores por campo sin borrar los datos.
500	Error del servidor	Mostrar mensaje general y revisar Laravel.
Network	Sin respuesta	Revisar backend, URL, CORS o red.

27. Paso 11 — Mostrar errores 422 junto a cada campo

```
catch (error) {
  if (error.status === 422) {
    setErrors(error.errors);
    return;
  }

  setGeneralError(error.message);
}

<input
  name="nombre"
  value={form.nombre}
  onChange={handleChange}
/>

{errors.nombre && (
  <small className="error">
    {errors.nombre[0]}
  </small>
)}
```

- No borre el formulario cuando la validación falla.
- Muestre el mensaje cerca del campo correspondiente.
- Use el mismo patrón visual en todos los formularios.
- La regla definitiva proviene del backend; React presenta el resultado de forma usable.

28. Paso 12 — Editar un registro

La edición combina una ruta con parámetro, carga del recurso, precarga del estado y una solicitud PATCH/PUT.

```
const { id } = useParams();

useEffect(() => {
  catalogosApi.get(id).then(item => {
    setForm({
      nombre: item.nombre,
      descripcion: item.descripcion ?? '',
      activo: item.activo,
    });
  });
}, [id]);

await request('/catalogos/' + id, {
  method: 'PATCH',
  body: JSON.stringify(form),
});

navigate('/catalogos');
```

29. Paso 13 — Eliminar con confirmación

Eliminar es una acción destructiva. Debe ser explícita, autorizada y confirmada.

```
async function eliminar(item) {
  const ok = await confirmar(item.nombre);
  if (!ok) return;

  await catalogosApi.remove(item.id);
  setItems(prev =>
    prev.filter(x => x.id !== item.id)
  );
}
```

```
);
}
```

- No ejecute DELETE inmediatamente ante un clic accidental.
- Muestre el nombre del registro y el efecto de la acción.
- Deshabilite el botón mientras la solicitud está en curso.
- Si Laravel responde 403, conserve el registro y muestre que la operación no fue autorizada.
- Si la API responde 204, no intente interpretar un cuerpo JSON vacío.

30. Paso 14 — Configurar la URL de la API con Vite

No escriba la dirección del backend repetida en cada archivo.

```
# .env
VITE_API_URL=http://127.0.0.1:8000/api

# service
const API_URL = import.meta.env.VITE_API_URL;
```

- Solo las variables con prefijo VITE_ se exponen al código cliente.
- No coloque contraseñas o claves privadas: los valores del navegador se pueden inspeccionar.
- Documente un .env.example sin secretos para que el equipo sepa qué configurar.

31. Paso 15 — Mantener permisos dentro del CRUD

React decide qué acciones mostrar, pero Laravel sigue protegiendo cada endpoint.

```
{can('catalogos.delete') && (
  <button onClick={() => eliminar(item)}>
    Eliminar
  </button>
)}

Route::delete('/catalogos/{catalogo}', ...)
->middleware('can:catalogos.delete');
```

Principio

Ocultar el botón no protege la API. Laravel debe responder 403 ante una acción no autorizada.

32. Trabajo de la Sección 6 sobre el proyecto

- ☐ Crear api.js y un servicio del módulo con list/get/create/update/remove.
- ☐ Convertir el formulario en controlado con estado inicial, handleChange y submit.
- ☐ Validar el backend mediante Form Request o validate().
- ☐ Implementar listado con loading, empty y error.
- ☐ Completar edición con ruta :id, precarga y PATCH/PUT.
- ☐ Completar eliminación con confirmación.
- ☐ Mostrar errores 422 junto a los campos.
- ☐ Aplicar permisos en la interfaz y comprobar 403 en el backend.

33. Pruebas mínimas de la Sección 6

Prueba	Acción	Resultado esperado
--------	--------	--------------------

P1 · Listar	Abrir el módulo.	loading → datos o empty.
P2 · Crear válido	Enviar datos correctos.	201 + registro persistido.
P3 · Crear inválido	Dejar requerido vacío.	422 + mensaje junto al campo.
P4 · Editar	Modificar un id existente.	Cambios persistidos.
P5 · Eliminar	Confirmar eliminación.	Registro desaparece tras éxito.
P6 · Sin permiso	Intentar crear/editar/eliminar.	403 y sin cambio de datos.

Herramientas de diagnóstico

Use la pestaña Network y la consola del navegador. Observe método, URL, status y respuesta JSON. En Laravel revise la terminal y, cuando corresponda, storage/logs/laravel.log.

34. Diagnóstico de errores frecuentes

Problema	Qué significa	Qué revisar
Failed to fetch	No hubo respuesta útil.	Backend, URL, CORS o red.
401	Sesión no válida.	Cookies/credenciales y flujo de autenticación.
403	Usuario autenticado sin autorización.	Permisos, policy o middleware.
404	Ruta o id incorrecto.	route:list y URL utilizada por el servicio.
422	Datos no cumplen reglas.	Objeto errors de la respuesta JSON.
500	Excepción del servidor.	Terminal y storage/logs/laravel.log.

Método de diagnóstico

Antes de cambiar código al azar, pregunte: ¿falló antes de salir de React, durante la red, en Laravel, en autorización, en validación o en la base de datos?

35. Entregable final de las Secciones 5 y 6

Al terminar ambas secciones, cada equipo debe demostrar que el módulo trabajado quedó organizado y conectado de extremo a extremo.

- ☐ Estructura de carpetas clara en src/.
- ☐ DashboardLayout con Sidebar, Header y Outlet.
- ☐ Rutas públicas, privadas, dinámicas y 404.
- ☐ Menú activo con NavLink.
- ☐ Permisos aplicados a navegación y acciones.
- ☐ CRUD dividido en página + componentes.
- ☐ Formulario controlado.
- ☐ Servicio HTTP reutilizable.
- ☐ Validación Laravel con respuesta 422.
- ☐ Listado con loading/empty/error.
- ☐ Crear, editar y eliminar funcionando.
- ☐ Variables de entorno para la URL de API.
- ☐ Pruebas de 401, 403, 404, 422 y camino exitoso.

Evidencia recomendada

Capturas de la estructura de carpetas, rutas, navegación, formulario, listado y pestaña Network; además, una explicación breve del recorrido de datos desde la interfaz React hasta Laravel y su respuesta de regreso al frontend.

36. Preguntas de preparación para la defensa

14. ¿Qué diferencia hay entre una SPA y una navegación con recarga completa?
15. ¿Por qué CatalogosPage debería estar separado de CatalogoTable y CatalogoForm?
16. ¿Qué diferencia existe entre props y state? Muestre un ejemplo del proyecto.
17. ¿Qué función cumplen BrowserRouter, Routes y Route?
18. ¿Para qué se utiliza <Outlet /> dentro del layout?
19. ¿Cómo obtiene una página el id de una ruta con :id?
20. ¿Por qué NavLink es útil en el menú?
21. ¿Qué convierte a un input en controlado por React?
22. ¿Por qué debe validarse en Laravel aunque React ya valide?
23. ¿Qué información suele contener una respuesta 422?
24. ¿Por qué centralizar fetch en una capa de servicios?
25. ¿Qué diferencia existe entre loading, saving y error?
26. Explique el recorrido desde Guardar hasta que el registro queda persistido.
27. ¿Por qué se revisa response.ok al usar fetch?
28. ¿Qué debe ocurrir si un usuario intenta DELETE y Laravel responde 403?

37. Lista de comprobación individual

Antes de presentar el proyecto, cada integrante debe ser capaz de marcar todos los puntos siguientes.

- ☐ Puedo mostrar en el código una ruta y explicar qué página renderiza.
- ☐ Puedo explicar qué hace el layout y dónde se usa Outlet.

- ☐ Puedo mostrar un componente que recibe props.
- ☐ Puedo mostrar un estado creado con useState.
- ☐ Puedo explicar una llamada a la API desde el servicio.
- ☐ Puedo localizar dónde se maneja un error 422.
- ☐ Puedo explicar por qué un botón puede ocultarse en React y aun así requerir autorización en Laravel.
- ☐ Puedo explicar el flujo completo de crear o editar un registro.
- ☐ Puedo usar Network para identificar método, URL, status y respuesta.
- ☐ Puedo explicar qué parte del trabajo realicé dentro del equipo.

Meta final

El proyecto debe quedar preparado para seguir agregando módulos con el mismo patrón: página + componentes + rutas + servicio + validación + permisos.